MULTIMODAL PDF RAG

Project Approach and Methodology

Assessment implementation report

Scope: Full-stack PDF question-answering application with page-linked evidence.

Evidence note: The assessment PDF was not supplied. This report describes implemented design and validation only; it does not claim assessment-document accuracy.

1. Project overview

The application provides an authenticated workspace for uploading PDFs, extracting page-aware content, indexing that content, and asking document-grounded questions. The browser client is built with React, TypeScript, Vite, and Tailwind CSS. The API uses FastAPI, SQLAlchemy, and Pydantic. PostgreSQL is the Docker deployment database; SQLite supports local testing. Qdrant stores embeddings and document ownership metadata. OpenAI is used for text embeddings, figure descriptions, and answer generation.

The relational database is the source of truth for users, documents, page text, extracted tables and figures, chunks, and chat messages. The PDF remains in private upload storage. Each API access to a document or chat is scoped to the authenticated owner.

2. Problem statement and design goals

PDFs mix prose, tabular values, scanned pages, and visual content. Plain text search can lose table structure and cannot interpret figures. The design therefore treats content types explicitly, preserves page numbers through ingestion, retrieves relevant evidence before generation, and returns structured citations. When no eligible evidence is found, the backend returns an explicit insufficient-evidence response.

Key goals

- Keep user documents and conversations isolated at every API lookup.
- Preserve source page references for text, table snippets, and figure descriptions.
- Make processing states and failures observable to the user.
- Treat extracted document content as untrusted evidence, never as instructions.
- Keep credentials in backend environment configuration and out of the frontend.

3. System architecture

Layer	Implementation and responsibility
Frontend	React + TypeScript SPA; account forms, document library, chat history, citations, and authenticated PDF preview.
API	FastAPI routes for authentication, documents, chat sessions/messages, health, pages, sources, tables, figures, and PDF streaming.
Relational persistence	SQLAlchemy models and Alembic migrations for users, documents, pages, chunks, tables, figures, sessions, and messages.
Vector retrieval	Qdrant cosine-vector collection with owner and document metadata filters; local persistent Qdrant mode is available outside Compose.
AI services	OpenAI embeddings, vision descriptions, and chat completions. API key is backend-only.
File storage	PDFs and extracted images stored beneath the configured upload directory, served only after owner authorization.

4. PDF ingestion methodology

Upload validation checks the PDF extension, the PDF file signature, and a configurable byte limit. A UUID is assigned and the source is written under a per-user document directory. A database record is created with a queued status, and background processing updates it to processing, completed, or failed. Failures are recorded and partial page/table/figure/chunk records and vector entries are cleaned up where possible.

Text extraction uses PyMuPDF page by page. Page numbers and extracted text are retained in the relational database. A text-length threshold detects pages likely to need OCR; OCR is attempted on pages containing embedded images. PaddleOCR and PaddlePaddle are optional dependencies. If OCR dependencies are missing or OCR fails, the document is marked failed with an explicit error rather than silently discarding the page.

Tables are extracted per page using pdfplumber. Non-empty rows are retained as JSON row data and rendered as delimited text, then added as distinct table content to chunking. This preserves basic row/column order, but complex merged cells and multi-page tables may require manual verification.

Embedded images are extracted from PDF pages, excluding images that occupy at least 80 percent of the page area. Extracted images are sent to a configured vision-capable model for concise descriptions. Descriptions retain their page number and are added to searchable content. Vector graphics, unembedded figures, and OCR/chart-value accuracy are not guaranteed.

Chunking and embeddings

Text is whitespace-normalized and split into overlapping character chunks (default 1,200 characters with 180 characters of overlap). Page numbers are carried into relational and Qdrant metadata. Text, table rows, and available figure descriptions are separately represented in the evidence text before embedding. Embeddings are sent in batches of 64; Qdrant stores vectors and owner/document/page/chunk identifiers. The actual configured model and key determine availability and cost.

5. Retrieval, answer generation, and citations

For a question, the backend obtains a query embedding and asks Qdrant for the nearest chunks filtered by the current owner and selected completed documents. Candidates below a relevance threshold are discarded. Retrieval is semantic vector search; the implementation does not currently include a separate keyword/hybrid index or learned reranker.

The answer prompt includes retrieved excerpts, conversation history, and a strict evidence-only policy. Excerpts and conversation history are explicitly marked as untrusted or non-evidence context. The model is instructed to ignore instructions embedded in the PDF, avoid outside knowledge, abstain when evidence is insufficient, and cite only the supplied source labels. Labels not present in the retrieved set are stripped. The API stores both sides of each turn and returns source IDs, page numbers, excerpts, and relevance scores.

Citation behavior

A citation refers to an actual retrieved chunk and its stored document/page metadata; the UI uses it to open the corresponding document page. The authenticated file route streams the original PDF inline for page navigation. Citations are retrieval-level evidence, not proof that every generated claim is fully supported. The system does not yet validate every claim independently.

6. Database and security decisions

User passwords are hashed before persistence and authentication uses signed bearer tokens. The signing key and OpenAI credentials are environment configuration. SQL queries for documents and sessions include the authenticated owner ID; inaccessible resources return not-found responses to avoid disclosing their existence. File streaming uses the same ownership check. Qdrant filters include owner and selected document IDs.

Uploads are size-limited, extension-checked, and signature-checked; stored filenames are sanitized and the actual storage name is generated server-side. CORS origins are configurable. Database records use foreign keys and cascades. Production hardening still requires strong deployment secrets, TLS, backups, rate limiting, malware scanning, durable asynchronous jobs, and a privacy/security review.

7. Assumptions and challenges

- The supplied assessment PDF is absent; meaningful accuracy testing needs the real file and verified expected answers.
- OpenAI services and a reachable Qdrant deployment are required for full ingestion and chat; no credentials were supplied.
- OCR dependencies are optional and can be platform-sensitive, especially for PaddlePaddle.
- PDF table layouts vary widely; extraction is structural but not guaranteed to preserve merged headers or reading order.
- Vision interpretation is bounded by the extracted embedded images and model output.
- The current background task mechanism is process-local and is not durable across service restarts.

8. Testing and evaluation status

Validation actually run in this workspace:

Check	Result
Backend pytest suite	9 tests passed.
Frontend production build and TypeScript check	Passed (`npm run build`).
Backend Python compilation	Passed (`compileall`).
Alembic migration and schema check	Migrations to head succeeded; no schema drift detected.
Docker Compose validation / end-to-end startup	Not run: this environment has a Docker CLI without the Compose command/plugin.
Live OpenAI / Qdrant and assessment PDF	Not tested: no API credential or assessment PDF supplied.

The backend tests cover account flows, owner isolation, upload validation, empty-evidence abstention/history, chunking, and a generated text-PDF processing fixture with embedding/index operations isolated. They do not establish OCR accuracy, table or figure correctness on the absent assessment PDF, retrieval precision, or live model quality.

Suggested evaluation

Use verified questions covering text facts, table lookup/comparison/arithmetic, figure labels and trends, missing facts, and cross-document isolation. Record retrieval relevance, answer correctness, page/excerpt citation correctness, extraction fidelity, abstention correctness, processing failures, model configuration, and human-verified ground truth. Treat every computed answer and visual interpretation as needing explicit source validation.

9. Limitations and future improvements

Current limitations

- Vector-only retrieval has no BM25/hybrid retrieval or reranking.
- There is no dedicated deterministic Python calculation layer for table arithmetic; numerical answers need careful verification.
- Table extraction does not currently perform advanced multi-page table reconstruction or confidence scoring.
- OCR is optional and may fail if the required runtime/model packages are unavailable.
- Only embedded raster images are considered for vision descriptions; vector diagrams and some full-page scans may be missed.
- Extracted figure image previews are not exposed as a dedicated image endpoint; the original PDF page can be previewed.
- There is no durable task queue, automatic retry policy, duplicate-file detection, or comprehensive rate limiting.
- No live AI/vector-service integration, Docker Compose startup, or assessment-PDF evaluation was possible in the current environment.

Recommended next steps

- Add OCR confidence reporting, scanned-page raster fallback, and processing retries using a durable queue.
- Add hybrid keyword/vector retrieval, reranking, and versioned chunk/embedding metadata.
- Add validated table arithmetic and source-cell-level citations; never calculate from ambiguous/missing values.
- Add explicit figure preview endpoints and chart extraction evaluation with human-verified labels.
- Add pagination, deletion/resynchronization recovery, rate limiting, malware scanning, and production observability.
- Run an integration test with real PostgreSQL, Qdrant, configured AI services, and the assessment PDF; publish measured results and failure cases.

10. Conclusion

The repository now contains an end-to-end implementation foundation spanning the frontend, account-scoped API, PDF ingestion and semantic retrieval, persistence, migrations, tests, containers, and documentation. The verified checks demonstrate local backend behavior and a successful frontend build, not production readiness or assessment-level answer quality. External service configuration and evaluation on the actual assessment document remain necessary before acceptance.